

고급통계분석 - Assignment01

20220866 문가영

2025-10-18

Problem 1.

```
set.seed(1)
n = 200
x = seq(0, 1, length.out = n)
y = sin(2*pi*x) + rnorm(n, sd = 0.15)
```

(a).

```
df = data.frame(x, y)
write.csv(df, file = 'data/problem1.csv', row.names = FALSE)
```

(d).

GitHub 저장소 링크: [20220866 문가영 GitHub 링크](#)

Problem 2.

(a).

버블 정렬 함수 구현:

```
# 버블 정렬
bubble_sort <- function(vec, decreasing = FALSE){
  n = length(vec) # 길이가 n인 입력 벡터

  # 오름차순 옵션
  if (decreasing == FALSE){
    for (i in 1:(n-1)){
      for (j in 1:(n-i)){
        if (vec[j] > vec[j+1]){
          temp = vec[j]
          vec[j] = vec[j+1]
          vec[j+1] = temp
        }
      }
    }
  }

  # 내림차순 옵션
  else{
    for (i in 1:(n-1)){
      for (j in 1:(n-i)){
        if (vec[j] < vec[j+1]){
          temp = vec[j]
          vec[j] = vec[j+1]
          vec[j+1] = temp
        }
      }
    }
  }
}
```

```

        vec[j] = vec[j+1]
        vec[j+1] = temp
    }
}
}
}
return(vec)
}

```

구현한 버블 정렬 알고리즘 실행 결과 확인:

```

set.seed(1)
x = runif(10)

print('***원본 벡터 (x)***')
## [1] "***원본 벡터 (x)***"

print(x)
## [1] 0.26550866 0.37212390 0.57285336 0.90820779 0.20168193
## [7] 0.89838968 0.94467527 0.66079779 0.62911404 0.06178627

# 1. 오름차순 정렬 (decreasing = FALSE)
x_asc = bubble_sort(x, decreasing = FALSE)
print('***1. 오름차순 결과***')
## [1] "***1. 오름차순 결과***"

print(x_asc)
## [1] 0.06178627 0.20168193 0.26550866 0.37212390 0.57285336
## [7] 0.62911404 0.66079779 0.89838968 0.90820779 0.94467527

# 2. 내림차순 정렬(decreasing = TRUE)
x_desc = bubble_sort(x, decreasing = TRUE)
print('***2. 내림차순 결과***')
## [1] "***2. 내림차순 결과***"

print(x_desc)
## [1] 0.94467527 0.90820779 0.89838968 0.66079779 0.62911404
## [7] 0.57285336 0.37212390 0.26550866 0.20168193 0.06178627

```

(b).

퀵 정렬 함수 구현:

```

# 퀵 정렬
quick_sort <- function(vec, decreasing = FALSE){

```

```

n = length(vec) # 길이가 n인 입력 벡터

# 재귀 종료 조건
if (n <= 1){
  return(vec)
}

# Pivot 분리
pivot <- vec[1]
rest <- vec[-1]

# 분할
# 오름차순 옵션
if (decreasing == FALSE){
  Left <- rest[rest <= pivot]
  Right <- rest[rest > pivot]
}

# 내림차순 옵션
else{
  Left <- rest[rest >= pivot]
  Right <- rest[rest < pivot]
}

# 재귀 호출
Left_sort <- quick_sort(Left, decreasing)
Right_sort <- quick_sort(Right, decreasing)

# 결합
return(c(Left_sort, pivot, Right_sort))
}

```

구현한 퀵 정렬 알고리즘 실행 결과 확인:

```

set.seed(1)
x = runif(10)

print('***원본 벡터 (x)***')
## [1] "***원본 벡터 (x)***"

print(x)
## [1] 0.26550866 0.37212390 0.57285336 0.90820779 0.20168193
## [2] 0.89838968
## [7] 0.94467527 0.66079779 0.62911404 0.06178627

# 1. 오름차순 정렬 (decreasing = FALSE)
x_asc = quick_sort(x, decreasing = FALSE)
print('***1. 오름차순 결과***')

```

```
## [1] "***1. 오름차순 결과***"

print(x_asc)

## [1] 0.06178627 0.20168193 0.26550866 0.37212390 0.57285336
0.62911404
## [7] 0.66079779 0.89838968 0.90820779 0.94467527

# 2. 내림차순 정렬(decreasing = TRUE)
x_desc = quick_sort(x, decreasing = TRUE)
print('***2. 내림차순 결과***')

## [1] "***2. 내림차순 결과***"

print(x_desc)

## [1] 0.94467527 0.90820779 0.89838968 0.66079779 0.62911404
0.57285336
## [7] 0.37212390 0.26550866 0.20168193 0.06178627
```

Problem 3.

(a).

수치 미분 함수 구현:

```
# 수치 미분
num_diff <- function(f, x, h= 1e-6, method){
  if (method == 'forward'){
    result <- (f(x+h) - f(x)) / h
    return(result)
  }
  else if (method == 'backward'){
    result <- (f(x) - f(x-h)) / h
    return(result)
  }
  else if (method == 'central'){
    result <- (f(x+h) - f(x-h)) / (2*h)
    return(result)
  }
  else{
    stop('Choose the method you want to use.')
  }
}
```

구현한 수치 미분 알고리즘 실행 결과 확인:

```
n <- 100
x <- seq(0, 2*pi, length.out = n)
```

```

# f 함수와 실제 도함수(f') 정의
f <- function(x){
  return(cos(x) - x)
}

f_prime <- function(x){
  return(-sin(x) - 1)
}

# 구현한 함수 적용
# 1. 전진차분
y_for <- num_diff(f, x, method= 'forward')

# 2. 후진차분
y_back <- num_diff(f, x, method= 'backward')

# 3. 중심차분
y_cen <- num_diff(f, x, method= 'central')

```

시각화:

```

library(ggplot2)

# 해석적 도함수 정의
y_prime <- f_prime(x)

# 시각화
# 1. 전진차분
df_for <- data.frame(x = x, analytic = y_prime, forward = y_for)

p1 <- ggplot(df_for, aes(x = x)) +
  geom_line(aes(y = analytic), color= 'black', size= 1.15) +
  geom_line(aes(y = forward), color= 'red', size = 0.9) +
  labs(title= '1. 전진차분 vs 해석적 도함수',
        x= 'x',
        y= 'f'(x)') +
  theme_minimal()

## Warning: Using `size` aesthetic for lines was deprecated in ggplot2
## 3.4.0.
## Please use `linewidth` instead.
## This warning is displayed once every 8 hours.
## Call `lifecycle::last_lifecycle_warnings()` to see where this
## warning was
## generated.

# 2. 후진차분
df_back <- data.frame(x= x, analytic = y_prime, backward = y_back)

p2 <- ggplot(df_back, aes(x = x)) +

```

```

geom_line(aes(y = analytic), color= 'black', size= 1.15) +
geom_line(aes(y = backward), color= 'green', size = 0.9) +
labs(title= '2. 후진차분 vs 해석적 도함수',
      x= 'x',
      y= 'f'(x)') +
theme_minimal()

# 3. 중심차분
df_cen <- data.frame(x= x, analytic = y_prime, central = y_cen)

p3 <- ggplot(df_cen, aes(x = x)) +
  geom_line(aes(y = analytic), color= 'black', size= 1.15) +
  geom_line(aes(y = central), color= 'blue', size = 0.9) +
  labs(title= '3. 중심차분 vs 해석적 도함수',
        x= 'x',
        y= 'f'(x)') +
  theme_minimal()

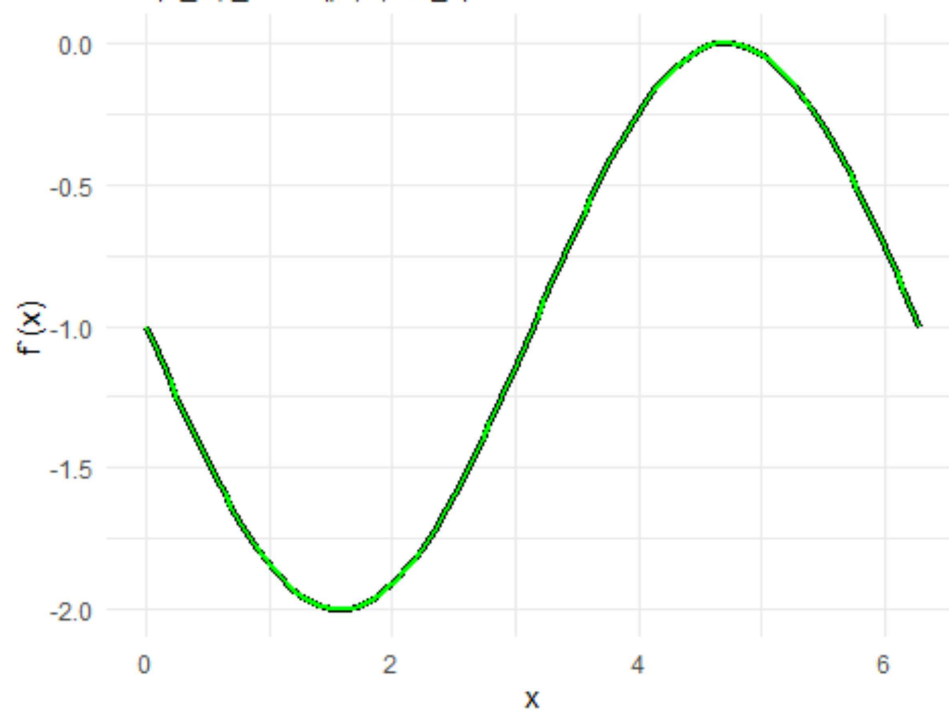
# plot 출력
print(p1)

```



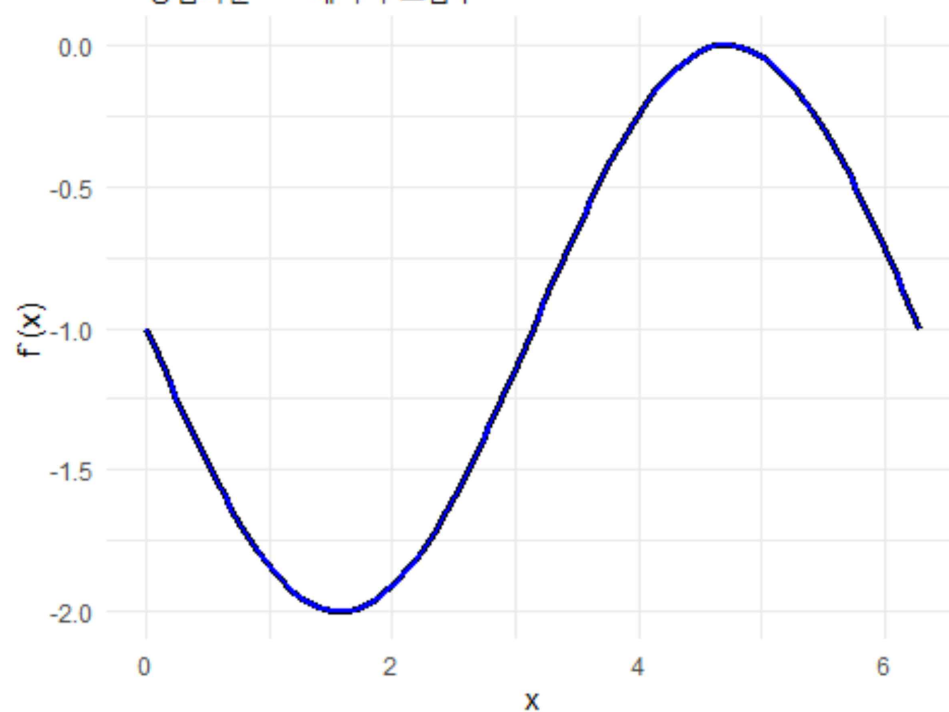
```
print(p2)
```

2. 후진차분 VS 해석적 도함수



```
print(p3)
```

3. 중심차분 VS 해석적 도함수



(b).

Newton-Rapshon 알고리즘 구현:

```
# Newton-Rapshon 방법
newton_rapshon <- function(f, fprime= NULL, x0, maxiter = 100, h = 1e-
6, epsilon = 1e-10){

  x_current <- x0
  iter <- 0

  # fprime 값이 입력되지 않은 경우
  if (is.null(fprime)){
    f_prime <- function(x_value){
      return((f(x_value+ h) - f(x_value - h)) / (2*h))
    }
  }

  # fprime 값이 입력된 경우
  else{
    f_prime <- fprime
  }

  # while문 사용해 반복 진행
  while(iter < maxiter){
    x_prev <- x_current

    # x_t 계산
    x_current = x_prev - f(x_prev) / f_prime(x_prev)
    iter <- iter + 1

    if (abs(x_current - x_prev) < epsilon){
      break
    }
  }
  return(x_current)
}
```

(c).

$f(x) = \cos(x) - x = 0$ 을 만족하는 해 찾기

- 수치미분 버전:

```
n <- 100
x <- seq(0, 2*pi, length.out = n)

# f 함수와 실제 도함수(f') 정의
f <- function(x){
  return(cos(x) - x)
}
```



```
newton_rapshon(f, NULL, 0.5)
```

```
## [1] 0.7390851
```

- 도함수 제공 버전:

```
# 도함수 제공
```

```
f_prime <- function(x){  
  return(-sin(x) - 1)  
}
```

```
newton_rapshon(f, f_prime, 0.5)
```

```
## [1] 0.7390851
```

Problem 4.

(a).

Left Rectangle 방식 구현:

```
left_rec <- function(f, a, b, n){  
  # 등간격 h 계산  
  h <- (b - a) / n  
  
  integral_sum <- 0  
  
  for (i in 0:(n-1)){  
    # 현재 좌측 끝점 위치 계산  
    x_i = a + (i * h)  
  
    integral_sum = integral_sum + f(x_i)  
  }  
  return(integral_sum * h)  
}
```

(b).

Trapezoid 방식 구현:

```
trapezoid <- function(f, a, b, n){  
  # 등간격 h 계산  
  h <- (b - a) / n  
  
  integral_sum <- f(a) + f(b)  
  
  for (i in 1:(n-1)){  
    # 현재 좌측 끝점 위치 계산
```

```

    x_i = a + (i * h)

    integral_sum = integral_sum + 2*f(x_i)
  }
  return((h / 2) * integral_sum)
}

```

(c).

Simpson 방식 구현:

```

simpson <- function(f, a, b, n){

  # n = 짝수
  if (n%%2 != 0){
    stop('n must be even.')
  }

  # 등간격 h 계산
  h <- (b - a) / n
  integral_sum <- f(a) + f(b)

  for(i in 1:(n-1)){
    # 현재 좌측 끝점 위치 계산
    x_i = a + (i * h)

    # i = odd인 경우
    if (i%%2 != 0){
      int_odd <- 4 * f(x_i)
      integral_sum <- integral_sum + int_odd
    }

    # i = even인 경우
    else{
      int_even <- 2 * f(x_i)
      integral_sum <- integral_sum + int_even
    }
  }
  return((h / 3) * integral_sum)
}

```

(d).

$f(x) = \sin(x)$ 인 경우의 수치적분 결과:

```

library(flextable)

f <- function(x){
  return(sin(x))
}
n <- 100

```

```
integral <- data.frame(
  '방법' = c('Left Rectangle', 'Trapezoid', 'Simpson'),
  '계산 결과' = c(left_rec(f, 0, pi, n), trapezoid(f, 0, pi, n),
    simpson(f, 0, pi, n))
)
flextable(integral)
```

방법	계산.결과
Left Rectangl e	1.99983 6
Trapezoi d	1.99983 6
Simpson	2.00000 0

(e).

해석적 값:

$$\int_0^{\pi} \sin(x) dx = [-\cos(x)]_0^{\pi} = -\cos(\pi) - (-\cos(0)) = 1 - (-1) = 2$$

시각화:

```
# 해석적 적분 값
f_true <- 2

# n 값 벡터
n_values <- c(10, 30, 60, 100, 150, 200)

f <- function(x){
  return(sin(x))
}

a<- 0; b<- pi

# 오차 계산
results <- data.frame(n = integer(), Method = character(), Error =
  numeric())
```

```

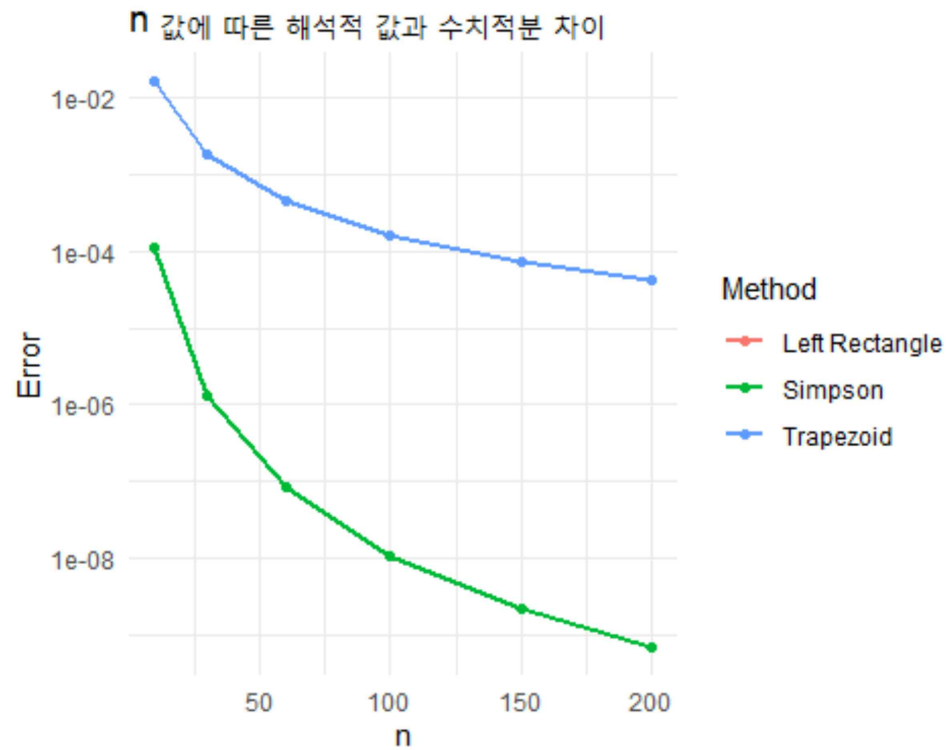
for (n in n_values){
  rec_val <- left_rec(f, a, b, n)
  trap_val <- trapezoid(f, a, b, n)
  simps_val <- simpson(f, a, b, n)

  rec_err <- abs(f_true - rec_val)
  trap_err <- abs(f_true - trap_val)
  simps_err <- abs(f_true - simps_val)

  results <- rbind(results,
                    data.frame(n = n, Method = 'Left Rectangle', Error
= rec_err),
                    data.frame(n = n, Method = 'Trapezoid', Error =
trap_err),
                    data.frame(n = n, Method = 'Simpson', Error =
simps_err))
}

# 시각화
library(scales)
p <- ggplot(results, aes(x = n)) +
  geom_line(aes(y = Error, color= Method), size = 1) +
  geom_point(aes(y = Error, color= Method), size = 1.5) +
  scale_y_log10(labels = scales::label_scientific()) +
  labs(title = 'n 값에 따른 해석적 값과 수치적분 차이', x= 'n', y=
'Error') +
  theme_minimal()
print(p)

```



Problem 5.

(a).

```
A <- matrix(c(4, 2, 2, 2, 5, 1, 2, 1, 3), 3)
U <- chol(A)
```

```
# L = U^T
L <- t(U)
```

```
print(L%*%t(L))
```

```
##      [,1] [,2] [,3]
## [1,]    4    2    2
## [2,]    2    5    1
## [3,]    2    1    3
```

```
print(A)
```

```
##      [,1] [,2] [,3]
## [1,]    4    2    2
## [2,]    2    5    1
## [3,]    2    1    3
```

(b).

forward 함수 구현:

```

forward <- function(L, y){
  # 행렬의 크기(n) 구하기
  n <- nrow(L)
  # 결과 벡터 z 초기화
  z <- numeric(n)

  for (i in seq_len(n)){

    if (i > 1){
      idx <- seq_len(i - 1)
      current_sum <- sum(L[i, idx] * z[idx])
    }
    else{
      current_sum <- 0
    }
    z[i] = (y[i] - current_sum) / L[i, i]
  }
  return(z)
}

```

(c).

backward 함수 구현:

```

backward <- function(L_t, z){
  # 행렬의 크기(n) 구하기
  n <- nrow(L_t)
  # 결과 벡터 x 초기화
  x <- numeric(n)

  for (i in n:1){
    if (i < n){
      idx <- (i + 1):n
      current_sum <- sum(L_t[i, idx] * x[idx])
    }
    else{
      current_sum <- 0
    }
    x[i] = (z[i] - current_sum) / L_t[i, i]
  }
  return(x)
}

```

(d).

forward / backward 함수 결과 확인:

```

# 행렬 A
A <- matrix(c(4, 2, 2, 2, 5, 1, 2, 1, 3), 3)

# 벡터 b

```

```

b <- c(1, -2, 3)

# 상삼각행렬(U)과 하삼각행렬(L)
U <- chol(A)
L <- t(U)

# 1. forward 함수 결과
z <- forward(L, b)
print(z)

## [1] 0.500000 -1.250000 1.767767

# forwardsolve와 비교
z_true <- forwardsolve(L, b)
all.equal(z, as.vector(z_true))

## [1] TRUE

# 2. backward 함수 결과
x <- backward(U, z)
print(x)

## [1] -0.0625 -0.6250 1.2500

# backsolve와 비교
x_true <- backsolve(U, z)
all.equal(x, as.vector(x_true))

## [1] TRUE

# solve(A, b) 결과
solve(A, b)

## [1] -0.0625 -0.6250 1.2500

```

Problem 6.

(a).

Gaussian Kernel 함수 구현:

```

gaussian_kernel <- function(x, x_prime, rho = 1){
  diff_sq <- (x - x_prime)^2
  result <- exp(- rho * diff_sq)
  return(result)
}

```

(b).

KRR 적합 함수 구현:

```

fit_krr <- function(X, y, lambda = 0.0001, rho = 1){
  # 데이터 크기(n) 확인
  n <- length(y)

  # 커널 행렬 K 계산
  K <- outer(as.numeric(X), as.numeric(X), function(a, b){
    exp(- rho * (a - b)^2)
  })

  # 정규화 항 계산:  $K + \lambda * I_n$ 
  K_reg <- K + lambda * diag(n)

  # 계수 alpha 계산:  $\alpha = (K + \lambda * I_n)^{-1} * y$ 
  alpha <- solve(K_reg, y)

  structure(list(alpha = alpha, X_train = X, y_train = y, lambda =
lambda, rho = rho), class = 'krr')
}

```

(c).

predict 함수 확장 구현:

```

predict.krr <- function(obj, X_new, ...){

  # 파라미터 추출
  alpha <- obj$alpha
  X_train <- as.numeric(obj$X_train)
  rho <- obj$rho

  sapply(X_new, function(x){
    k_x_X <- exp(- rho * (x - X_train)^2)
    sum(k_x_X * alpha)
  })
}

```

(d).

plot 함수 구현:

```

plot.krr <- function(x, ...) {

  # 필요한 데이터 추출 및 예측 곡선 생성
  model <- x

  # 훈련 데이터 x 범위 내에서 부드러운 곡선을 위한 새로운 x 벡터 생성
  X_train <- model$X_train
  y_train <- model$y_train

  # X_train의 최소/최대값 기준으로 100개의 새로운 x 값 생성

```



```

X_new <- seq(min(X_train), max(X_train), length.out = 100)

# 예측값 계산
Y_pred <- predict.krr(model, X_new)

# 시각화를 위한 데이터프레임 생성
df_train <- data.frame(X = X_train, Y = y_train)
df_pred <- data.frame(X = X_new, Y = Y_pred)

# 시각화
library(ggplot2)

p <- ggplot() +
  geom_point(data = df_train, aes(x = X, y = Y),
             color = 'blue', size = 3, alpha = 0.7) +
  geom_line(data = df_pred, aes(x = X, y = Y),
            color = 'red', size = 1) +
  labs(
    title = paste0('Kernel Ridge Regression 예측 곡선 ( $\lambda =$ ',
model$lambda, ',  $\rho =$ ', model$rho, ')'),
    x = 'X',
    y = 'Y'
  ) +
  theme_minimal()

print(p)
}

```

(e).

```

set.seed(1)
n = 150
X = matrix(runif(n, -1, 1), ncol = 1)
ftrue = function(x) sin(2*pi*x) + 0.5 * cos(4*pi*x)
y = ftrue(X[, 1]) + rnorm(n, sd = 0.1)

# 모델 적합
krr_model <- fit_krr(X, y)

# 예측 및 시각화
plot(krr_model)

```

Kernel Ridge Regression 예측 곡선 ($\lambda=1e-04$, $\rho=1$)

